



S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Post Lab

Aim: Write a C program to implement and compare different Disk Scheduling Algorithms such as FCFS, SSTF, and SCAN, and determine the total head movement for each algorithm using a given disk request queue.

Name: Neesha Tabhane

USN:CM24026

Semester / Year:4th/2nd

Academic Session:2025-26

Date of Performance:

Date of Submission:

❖ **Aim:**

Write a C program to implement and compare different Disk Scheduling Algorithms such as FCFS, SSTF, and SCAN, and determine the total head movement for each algorithm using a given disk request queue.

❖ **Objectives:**

- a) To understand the concept of **disk scheduling** in operating systems.
- b) To implement **FCFS, SSTF, and SCAN** disk scheduling algorithms using C.
- c) To calculate and compare the **total head movement** of different algorithms.
- d) To analyze which disk scheduling algorithm provides better performance.
- e) To study how efficient disk head movement improves system performance.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum **1 GHz**
- RAM: **512 MB** or higher
- Storage: **100 MB** free space

✓ **Software Requirements:**

- Operating System: **Linux / Unix / Windows**
- Compiler: **GCC / Turbo C / CodeBlocks / Dev C++**
- Text Editor: **VS Code / Nano / Vim / Notepad++**

Theory: Disk Scheduling (SCAN Algorithm)

Disk scheduling is a crucial part of the **Operating System**, responsible for managing how disk I/O requests are handled efficiently. Since disk access is much slower compared to CPU operations, optimizing disk performance becomes very important for improving overall system efficiency.

When multiple processes request disk access simultaneously, these requests are stored in a queue. The operating system must decide the sequence in which these requests are serviced. This decision is made using various disk scheduling algorithms.

❖ **Need for Disk Scheduling**

Disk scheduling is required because:

- Disk operations involve **mechanical movement** of the read/write head.
- Large movement of disk head increases **seek time**.
- Inefficient scheduling leads to **poor system performance**.
- Multiple processes generate I/O requests at the same time.

Therefore, disk scheduling algorithms are designed to:

- Minimize **seek time**
- Reduce **total head movement**

- Increase **throughput**
 - Improve **response time**
-

❖ **Structure of a Disk**

A disk consists of:

- **Platters:** Circular disks where data is stored.
- **Tracks:** Concentric circles on each platter.
- **Sectors:** Divisions of tracks where data is stored.
- **Read/Write Head:** Moves across tracks to access data.
- **Cylinder:** Set of tracks aligned vertically across platters.

The disk head moves across these tracks to access requested data, and this movement contributes to the total delay.

❖ **Disk Access Time**

The time required to access data from disk is called **Disk Access Time**.

It consists of:

1. **Seek Time**
Time required to move the disk head to the desired track.
2. **Rotational Latency**
Time taken for the required sector to rotate under the head.
3. **Transfer Time**
Time required to transfer the data.

Formula:

$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

❖ **Types of Disk Scheduling Algorithms**

Several disk scheduling algorithms are used in operating systems:

1. FCFS (First Come First Serve)

- Processes requests in the order they arrive.
- Simple but inefficient.

2. SSTF (Shortest Seek Time First)

- Selects the request closest to current head position.
- Reduces seek time but may cause starvation.

3. SCAN (Elevator Algorithm)

- Moves in one direction servicing all requests.
- Reverses direction at the end.

4. C-SCAN (Circular SCAN)

- Moves in one direction only.
- Jumps back to the beginning after reaching the end.

5. LOOK Algorithm

- Similar to SCAN but does not go till the end of disk.
- Stops at last request.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int request[] = {98, 183, 41, 122, 14, 124, 65, 67};
    int n = 8;
    int head = 53;
    int total = 0;
    int disk_size = 200;
    int left[20], right[20];
    int l = 0, r = 0;

    printf("Initial Head Position: %d\n", head);
    printf("Request Queue: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", request[i]);
    }
    printf("\n");

    // Separate requests into left and right of head
    for(int i = 0; i < n; i++) {
        if(request[i] < head)
            left[l++] = request[i];
        else
            right[r++] = request[i];
    }

    // Sort left array
    for(int i = 0; i < l - 1; i++) {
        for(int j = i + 1; j < l; j++) {
            if(left[i] > left[j]) {
                int temp = left[i];
                left[i] = left[j];
                left[j] = temp;
            }
        }
    }
}
```

```
}  
}
```

```
// Sort right array  
for(int i = 0; i < r - 1; i++) {  
    for(int j = i + 1; j < r; j++) {  
        if(right[i] > right[j]) {  
            int temp = right[i];  
            right[i] = right[j];  
            right[j] = temp;  
        }  
    }  
}
```

```
printf("\nOrder of Execution (SCAN):\n");
```

```
// Move towards right first  
for(int i = 0; i < r; i++) {  
    printf("%d -> %d (distance = %d)\n", head, right[i], abs(head - right[i]));  
    total += abs(head - right[i]);  
    head = right[i];  
}
```

```
// Move to end of disk  
printf("%d -> %d (distance = %d)\n", head, disk_size - 1, abs(head -  
(disk_size - 1)));  
total += abs(head - (disk_size - 1));  
head = disk_size - 1;
```

```
// Reverse direction and service left side  
for(int i = l - 1; i >= 0; i--) {  
    printf("%d -> %d (distance = %d)\n", head, left[i], abs(head - left[i]));  
    total += abs(head - left[i]);  
    head = left[i];  
}  
printf("\nTotal Head Movement = %d\n", total);  
return 0;}
```

OUTPUT:

```
Output
Initial Head Position: 53
Request Queue: 98 183 41 122 14 124 65 67

Order of Execution (SCAN):
53 -> 65 (distance = 12)
65 -> 67 (distance = 2)
67 -> 98 (distance = 31)
98 -> 122 (distance = 24)
122 -> 124 (distance = 2)
124 -> 183 (distance = 59)
183 -> 199 (distance = 16)
199 -> 41 (distance = 158)
41 -> 14 (distance = 27)

Total Head Movement = 331
```

❖ Conclusion:

In this practical, we implemented the **SCAN Disk Scheduling Algorithm** and calculated the total head movement of the disk head.

The SCAN algorithm services requests in one direction and then reverses its path, which reduces unnecessary movement compared to FCFS. Hence, it provides better efficiency and improved disk performance.

❖ Discussion Questions

1. What is disk scheduling?
2. Why is SCAN called the Elevator Algorithm?
3. What is the main advantage of SCAN over FCFS?
4. What is seek time?
5. What is the drawback of SCAN?

Date :

Signature

Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26